The Four Gateways: Setup Guide

Run the Flight Recorder demo and the four gateways labs on your own Mac or Windows laptop. DAMA Rocky Mountain, October 2026. Nathan Platter, Lincoln NE.

SYNTHETIC SAMPLE DATA ONLY. Every name, number, and ID in this kit is invented. In mock mode and with a local model the labs run fully offline, and nothing leaves your laptop. The optional cloud model mode (section 7) sends redacted synthetic data to the provider you choose.

What you get

- Clickable demo: flight-recorder/index.html. Add ?gateways=1 to turn on the four gateways view.
- Python labs, one per gateway, in dama-2026/gateways/. They run in mock mode with no model installed.
- Optional local model through Ollama: llama3.1:8b or deepseek-r1:7b.
- Optional cloud model with your own API key: Anthropic Claude (Opus by default), xAI Grok, OpenAI, or any OpenAI-compatible server. Paid per use.
- agents/AGENTS.md and agents/guardrails.yaml: the same four gates written for an agent.

The four gateways at a glance

Gateway	Control	Lab	Demo step
G1. Use	Classify, redact, keep sensitive data on a local model	g1_classify_redact.py	Step 2
G2. Prompt	Locked instructions, injection defense, tool allow-list	g2_prompt_guard.py	Steps 1 and 5
G3. Context	Permissioned retrieval, minimum data, source trace	g3_context_scope.py	Steps 2 and 4
G4. Self-check and exit	Fail closed, named human, audit log	g4_selfcheck_exit.py	Step 3, Closer

1. Prerequisites

- Mac: macOS 14 Sonoma or newer for Ollama. Apple silicon (M1 or later) runs local models best.
- Windows: Windows 10 22H2 or newer, or Windows 11.

- 16 GB of RAM recommended for 7B or 8B models (8 GB works with smaller models such as llama3.2:3b).
- About 10 GB of free disk space for two models.
- A modern browser: Safari, Chrome, Edge, or Firefox.
- Python 3.10 or newer. The labs use only the standard library, so there is nothing to pip install.
- Git is optional. You can download a ZIP instead.

Check Python:

```
Mac:     python3 --version
Windows:  py --version
```

If Python is missing: on Mac, install it from python.org/downloads (or run xcode-select --install). On Windows, install it from python.org/downloads and tick "Add python.exe to PATH", or run:

```
winget install Python.Python.3.12
```

2. Install Ollama and pull the models (optional)

Skip this section if you only want mock mode. Mock mode shows every gateway with repeatable output.

Mac

- Download Ollama for macOS from ollama.com/download.
- Open the .dmg and drag Ollama into Applications. Start it once. It offers to add the ollama command to your PATH; accept.

Windows

- Download OllamaSetup.exe from ollama.com/download and run it. No administrator rights are needed.
- Ollama runs in the background, and the ollama command works in PowerShell or Command Prompt.

Pull the models (both platforms):

```
ollama pull llama3.1:8b      # about 4.9 GB
ollama pull deepseek-r1:7b   # about 4.7 GB
ollama list
```

Quick test: ollama run llama3.1:8b "Say hello in five words." The local API listens on http://localhost:11434.

3. Get the code

With Git:

```
git clone https://github.com/nathanplatteruser/dama-rocky-mountain-2026.git
cd dama-rocky-mountain-2026
```

Without Git: open github.com/nathanplatteruser/dama-rocky-mountain-2026, click Code, then Download ZIP. Unzip it and open a terminal in the unzipped folder.

4. Optional virtual environment

The labs need no packages, but a virtual environment keeps your experiments separate if you add any.

```
Mac:      python3 -m venv .venv && source .venv/bin/activate
Windows:  py -m venv .venv
          .venv\Scripts\Activate.ps1
```

If PowerShell blocks the activate script, run: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

5. Run in mock mode (no model)

```
cd dama-2026
Mac:      python3 run_all.py
Windows:  py run_all.py
```

You should see four sections and a summary ending in four PASS lines:

```
PASS G1 no sensitive value leaves
PASS G2 injection never moves money
PASS G3 out-of-role doc denied
PASS G4 fails closed, writes nothing
```

Run the tests: python3 -m unittest -v test_gateways (Windows: py -m unittest -v test_gateways). Eleven tests should pass.

6. Run with a local model

```
Mac:
  FR_BACKEND=ollama FR_MODEL=llama3.1:8b python3 run_all.py
  FR_BACKEND=ollama FR_MODEL=deepseek-r1:7b python3 run_all.py

Windows PowerShell:
  $env:FR_BACKEND="ollama"; $env:FR_MODEL="llama3.1:8b"; py run_all.py
```

The model now writes the drafts. The gates still decide. A model may answer differently from the mock, so read each section: G2 blocks any tool not on the allow-list, G3 removes any sentence without a valid source, and G4 refuses to write a number when sources disagree. DeepSeek-R1 prints its reasoning; the labs strip it before checking. On a laptop without a GPU, expect 10 to 60 seconds per call.

7. Cloud model option (optional, paid)

Two versions, both run live, same labs: the free local version (Ollama on your laptop) and the paid cloud version (Claude, Grok, or ChatGPT with your own API key). Same prompts, same four gateways, same audit log. Only the model changes.

Backend	Provider	Key	Default model	Cost
mock	None, scripted	None	Scripted	Free
ollama	Your laptop	None	llama3.1:8b	Free
anthropic	Anthropic Claude	ANTHROPIC_API_KEY	claude-opus-5-5	Paid, about 5 to 15 cents a run
xai	xAI Grok	XAI_API_KEY	grok-4.7	Paid, about 1 to 5 cents a run
openai	OpenAI	OPENAI_API_KEY	gpt-6.1-sol	Paid, about 2 to 8 cents a run
compatible	OpenAI-compatible server	FR_API_KEY	Set FR_MODEL	Depends on the server

Pick one: free local or paid cloud

- Free local: mock mode (scripted, no install) or Ollama (a local model, section 6). No account, no key, nothing leaves the laptop. Cost: $0. Pick this if you can install Ollama, or if your laptop has 8 GB of RAM or more and you want to keep everything offline.
- Paid cloud: anthropic (Claude Opus), xai (Grok), or openai (ChatGPT models). Pick this if you cannot install Ollama, your laptop is slow, or you want a frontier model. You need an account with the provider, a payment method, and an API key; you pay the provider per token. A full run is four calls, about 500 tokens in plus the reply and the model's thinking. Rough cost per full run: Claude Opus 5.5 about 5 to 15 cents; Grok 4.7 about 1 to 5 cents; GPT-6.1 Sol about 2 to 8 cents; GPT-6 Astra about 10 to 40 cents. Prices as of October 2026; check the provider's pricing page.
- compatible: any OpenAI-compatible server, such as OpenRouter, Azure, LM Studio, or vLLM. Cost depends on the server; a local server is free.

Step 1: Get a key

- Anthropic: console.anthropic.com, then API Keys. Set ANTHROPIC_API_KEY.
- xAI: console.x.ai, then API Keys. Set XAI_API_KEY.
- OpenAI: platform.openai.com, then API keys. Set OPENAI_API_KEY.
- Keys are read from the environment at call time, sent only to the provider you picked, and never printed, logged, or saved by the labs. A key set this way lasts only for that terminal

window; open a new window and set it again. Never put a key in a file in this folder, and never commit one to Git.

Step 2: Run it

```
Mac (Terminal):
  export ANTHROPIC_API_KEY=your-key
  FR_BACKEND=anthropic python3 run_all.py
  FR_BACKEND=anthropic FR_MODEL=claude-sonnet-5-5 python3 run_all.py
  export XAI_API_KEY=your-key
  FR_BACKEND=xai python3 run_all.py
  export OPENAI_API_KEY=your-key
  FR_BACKEND=openai python3 run_all.py

Windows (PowerShell):
  $env:ANTHROPIC_API_KEY="your-key"
  $env:FR_BACKEND="anthropic"
  py run_all.py
  # Grok:   $env:XAI_API_KEY="your-key";    $env:FR_BACKEND="xai"
  # OpenAI: $env:OPENAI_API_KEY="your-key"; $env:FR_BACKEND="openai"

Windows (Command Prompt):
  set ANTHROPIC_API_KEY=your-key
  set FR_BACKEND=anthropic
  py run_all.py

OpenAI-compatible server:
  export FR_API_KEY=your-key
  FR_BACKEND=compatible FR_BASE_URL=https://openrouter.ai/api/v1 \
    FR_MODEL=model-name python3 run_all.py
```

Step 3: Switch back

Free local model: set FR_BACKEND=ollama. Mock mode: remove FR_BACKEND (Mac: unset FR_BACKEND; PowerShell: Remove-Item Env:FR_BACKEND; Command Prompt: set FR_BACKEND=). FR_MODEL picks any model your account can use; the defaults are Claude Opus 5.5 (claude-opus-5-5), Grok 4.7 (grok-4.7), and GPT-6.1 Sol (gpt-6.1-sol; try gpt-6-astra for the OpenAI flagship). Cloud models think before they answer, so expect about 5 to 30 seconds per call.

Compare local and cloud side by side

Every line in out/flight-recorder.jsonl now names the backend and model that ran. Run once with Ollama, copy the log, run once with a cloud model, and compare. The events line up one for one. A cloud run adds one "G1 CLOUD EGRESS CHECKED" line before each model call. The models write different drafts; the gates decide the outcome either way.

Why this is a Gateway 1 decision

A cloud run sends data to the provider. Here that data is SYNTHETIC SAMPLE DATA and already redacted. Before every call the code scans the outgoing text and refuses to send if any raw sensitive value (SSN, phone, email, name, date of birth, address, health note) is present. On the OpenAI and xAI Responses API it also asks the provider not to store the request (store: false). In your own systems, check the provider agreement for retention, training use, and region before any real data goes to a cloud model.

Test the cloud path with no key and no internet: From the repository root, python3 tests/test_cloud.py runs every lab against a bundled stand-in for the Anthropic, Responses, and Chat Completions APIs, with the same misbehaving model the Ollama test uses. Add --real anthropic (or --real xai, --real openai) for one real, paid run. With make: make test-cloud.

8. Walk through each lab

G1. Use

Classifies each field of a synthetic applicant record, redacts the sensitive ones, and routes the raw record to local-only. Try it: add a new field with a phone number in free text and watch it get caught.

```
python3 gateways/g1_classify_redact.py
```

G2. Prompt

A broker note hides "ignore previous instructions" and asks for a $4,800 refund. The lab flags the injection and the code gate blocks move_money even if the model proposes it. Try it: add a tool to the allow-list and see what changes.

```
python3 gateways/g2_prompt_guard.py
```

G3. Context

Retrieves only documents the underwriting role may read (the HR file is denied), then removes any sentence whose number does not appear in its cited line. Try it: ask as the reinsurance role.

```
python3 gateways/g3_context_scope.py
```

G4. Self-check and exit

Three sources disagree on a treaty claim and one tries to instruct the AI. The lab discards the model's guess, writes no number, and sends one specific question to a named analyst. Every step lands in out/flight-recorder.jsonl.

```
python3 gateways/g4_selfcheck_exit.py
```

9. Open the clickable demo

- Open flight-recorder/index.html in your browser (double-click works), or use the live link: nathanplatteruser.github.io/dama-rocky-mountain-2026/flight-recorder/
- Click "Four gateways" in the top bar to turn the overlay on or off.
- Step 2: use the raw vs redacted toggle. Step 3: the fail-closed record hands a question to a human. Closer: download the evidence pack.

10. Troubleshooting

- **"python3" or "py" not found:** Install Python (section 1). On Windows, reopen the terminal after install so PATH updates.
- **"Could not reach Ollama at http://localhost:11434":** Start the Ollama app, or run ollama serve in another terminal. Or unset FR_BACKEND to go back to mock mode.
- **"model not found":** Run ollama pull with the exact name in FR_MODEL, then ollama list.
- **Very slow or out of memory:** Close other apps, or use a smaller model: ollama pull llama3.2:3b and set FR_MODEL=llama3.2:3b.
- **Port 11434 already in use:** Ollama is already running. That is fine.
- **PowerShell will not activate the venv:** Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, or skip the venv; it is optional.
- **Demo page looks blank:** Use a current browser. The demo is one HTML file with no downloads.
- **Refusing to send data off this machine:** OLLAMA_HOST points somewhere other than localhost. The labs only talk to a local model by design.
- **Test the Ollama path without downloading a model:** from the repository root, python3 tests/test_ollama.py starts a small stand-in for the Ollama API and runs every lab as llama3.1:8b and deepseek-r1:7b, including a "bad" model that obeys the injection, invents numbers, and echoes an SSN. All gates should still pass. Add --real llama3.1:8b to also run against your real Ollama. With make installed: make test-ollama.
- **A small model gives odd answers:** Expected. The gates decide the outcome, not the model. A model that forgets to cite sources gets its sentences removed by G3, which is the gate working.
- **"needs an API key": Set the key variable named in the message in the same terminal, or remove FR_BACKEND to use mock mode.**
- **HTTP 401 or 403 from a cloud provider: The key was rejected or cannot use that model. Make a new key, or pick another model with FR_MODEL.**
- **HTTP 404 from a cloud provider: The model name is wrong for your account. Set FR_MODEL to a model listed in the provider's console.**
- **HTTP 429 from a cloud provider: Rate limit or no credits left. Add credits or wait a minute.**
- **"Refusing to send: the outgoing text contains sensitive data": Gateway 1 working. Something unredacted was about to leave the machine, so nothing was sent.**

11. Data handling

- Every record in this kit is synthetic and labeled SYNTHETIC SAMPLE DATA.
- Mock mode makes no network calls. Ollama mode calls only localhost; the code refuses any other host. Cloud mode calls only the provider you picked, sends only redacted synthetic data, and refuses to send any raw sensitive value.
- The demo runs in your browser and uploads nothing. Intake answers are not saved.
- The audit log out/flight-recorder.jsonl stays in your folder. Delete it any time.
- Do not load real customer data into a teaching kit. Use it to design your own controls, then build them into your governed systems.

12. Use it in your own product

- Rewrite agents/AGENTS.md and agents/guardrails.yaml for your agent.
- Port the four checks: classify and redact, gate, retrieve with trace, reconcile and record.
- Keep one test per rule so the policy and the code cannot drift apart.

MIT license. Teaching example, not legal advice. Questions: nathanplatter@gmail.com